

Non-Uniform Time Grid: Refining the First and Last Cell

Optimal Transport / Schrödinger Bridge Project

Abstract

This note specifies the simplest non-trivial non-uniform time grid we consider for the 1D Python solver: take the uniform n_t -cell grid on $[0, 1]$ and split only the first and last cell in half. It sets out the two grids the LADMM splitting actually uses – a staggered grid carrying the hard Fokker–Planck (FP) constraint, and a collocated grid carrying the kinetic-energy (KE) prox – and derives the full discrete FP constraint on the refined-ends grid, row by row through both junctions. The surrounding solver (`operators.nu.py`, `projection.nu.py`, `prox.nu.py`, `pipeline.nu.py`) is already generic over any strictly increasing edge sequence, so this grid requires no new solver code, only a new grid constructor – implemented as `refine_ends_time_grid` in `time_grid.py` and regression-checked in `scripts/validate.nu.py` (5/5 checks pass, including this grid).

1 Two grids in the LADMM split

The LADMM splitting (`pipeline.nu.py`) carries two copies of the state, on two different grids, tied together by a consensus map:

- **Constraint grid** (code: the x -variable (μ, ψ)). This is where the FP constraint is enforced *exactly*, via the projection `proj_fokker_planck_banded_nu`. Staggered: density at interior time edges $\times$ space cell centers,

$$\mu(t_j, \xi_k), \quad j = 1, \dots, n_t^{\text{ref}} - 1, \quad k = 1, \dots, n_x,$$

and momentum at time cell centers $\times$ interior space nodes,

$$\psi(\tau_i, x_k), \quad i = 0, \dots, n_t^{\text{ref}} - 1, \quad k = 1, \dots, n_x - 1,$$

with $\mu(t_0, \cdot) = \rho_0$, $\mu(t_{n_t^{\text{ref}}}, \cdot) = \rho_1$ (fixed marginals) and $\psi(\cdot, x_0) = \psi(\cdot, x_{n_x}) = 0$ (no-flux).

- **KE grid** (code: the y -variable (ρ, m)). This is where the kinetic-energy prox `prox_ke_cc_nu` acts, pointwise in $m^2/(2\rho)$, so ρ and m must be *collocated* – both at cell centers, both axes:

$$\rho(\tau_i, \xi_k), \quad m(\tau_i, \xi_k), \quad i = 0, \dots, n_t^{\text{ref}} - 1, \quad k = 1, \dots, n_x.$$

Space is uniform throughout (n_x cells, $\Delta x = 1/n_x$, edges $x_k = k\Delta x$, centers $\xi_k = (k - \frac{1}{2})\Delta x$) – only the time axis is refined here, and only at the ends.

The consensus map

`A_fn` moves (μ, ψ) onto the KE grid's locations:

$$\hat{\rho}(\tau_i, \xi_k) = \frac{1}{2}[\mu(t_i, \xi_k) + \mu(t_{i+1}, \xi_k)], \quad \hat{m}(\tau_i, \xi_k) = \frac{1}{2}[\psi(\tau_i, x_{k-1}) + \psi(\tau_i, x_k)], \quad (1)$$

and the LADMM iteration drives $\hat{\rho} \rightarrow \rho$, $\hat{m} \rightarrow m$ while μ, ψ stays exactly FP-feasible and ρ, m stays exactly KE-optimal. Note $\hat{\rho}$ only interpolates *in time* (both sides share the same ξ_k) and $\hat{m}$ only interpolates *in space* (both sides share the same τ_i) – ψ 's time location is τ_i on *both* grids already, so refining the time grid never touches the momentum's time placement, only the density's.

Lemma 1 (Midpoint interpolation is exact regardless of cell width). *Let $t_i < t_{i+1}$ be any two grid edges and $\tau_i := \frac{1}{2}(t_i + t_{i+1})$ their midpoint (as every cell center in this codebase is, by construction – `time_grid.time_grid_from_edges`). For any values v_i, v_{i+1} , let $L(t)$ be the unique affine function with $L(t_i) = v_i$, $L(t_{i+1}) = v_{i+1}$. Then $L(\tau_i) = \frac{1}{2}(v_i + v_{i+1})$, independent of $\Delta t_i = t_{i+1} - t_i$.*

Proof. $L(t) = v_i + (v_{i+1} - v_i) \frac{t - t_i}{\Delta t_i}$. At $t = \tau_i$, $t - t_i = \Delta t_i/2$, so $L(\tau_i) = v_i + \frac{1}{2}(v_{i+1} - v_i) = \frac{1}{2}(v_i + v_{i+1})$. $\square$

So the plain average in (1)'s $\hat{\rho}$ formula is exactly the value of the linear interpolant of μ at the cell center, for any Δt_i – this is the reason `interp_t_at_phi` needs no change on a non-uniform grid at all (`operators_nu.py` reuses it verbatim from `operators.py`), and it is the key fact behind the derivation in Section 3.

2 The refined-ends grid

Take the uniform n_t -cell grid and split *only* cell 0 ($[t_0, t_1]$) and cell $n_t - 1$ ($[t_{n_t-1}, t_{n_t}]$) in half. Everything in between is untouched:

edges	0	$h/2$	h	$2h$	$3h$	$\dots$	$(n_t-1)h$	$(n_t-1)h + h/2$	1
cell width	$h/2$		h				$h/2$		

So, writing $h = 1/n_t$ for the *base* resolution being refined,

$$\mathbf{t} = [0, \frac{h}{2}, h, 2h, 3h, \dots, (n_t-1)h, (n_t-1)h + \frac{h}{2}, 1], \quad (2)$$

i.e. one extra edge inserted into the first cell and one into the last. This has $n_t + 3$ edges, hence

$$n_t^{\text{ref}} = n_t + 2 \quad \text{cells}, \quad \Delta t_i = \begin{cases} h/2, & i \in \{0, 1, n_t^{\text{ref}}-2, n_t^{\text{ref}}-1\}, \\ h, & \text{otherwise,} \end{cases} \quad (3)$$

and $\sum_i \Delta t_i = 2 \cdot \frac{h}{2} + (n_t - 2)h + 2 \cdot \frac{h}{2} = n_t h = 1$, as required. Cell centers and $\Delta t_{\text{ref}} := 1/n_t^{\text{ref}}$ follow the usual construction from $\mathbf{t}$ (Eq. 2); nothing else about the grid changes. This affects *both* grids of Section 1 identically: the KE grid's τ_i are exactly the constraint grid's cell centers, so both move together, and only μ (not ψ) picks up any new time-interpolation behavior, per the remark above.

2.1 Worked example: $n_t = 6$

Here $h = 1/6$, $n_t^{\text{ref}} = 8$.

i (cell)	0	1	2	3	4	5	6	7
Δt_i	1/12	1/12	1/6	1/6	1/6	1/6	1/12	1/12
τ_i (center)	1/24	1/8	1/4	5/12	7/12	3/4	7/8	23/24

j (edge)	0	1	2	3	4	5	6	7	8
t_j	0	1/12	1/6	1/3	1/2	2/3	5/6	11/12	1

μ 's interior nodes are $t_1, \dots, t_7$ (plus $\mu(t_0, \cdot) = \rho_0$, $\mu(t_8, \cdot) = \rho_1$); ψ and the collocated (ρ, m) all live at the 8 cell centers $\tau_0, \dots, \tau_7$ above. (Verified against `refine_ends_time_grid(6)` numerically – matches to machine precision.)

3 The discrete FP constraint on this grid

The continuous constraint is $\partial_t \rho + \partial_x m = \varepsilon \partial_x^2 \rho$. Discretely, the residual is evaluated on the constraint grid but *landed* at the KE grid's locations (τ_i, ξ_k) (where ϕ , the projection's dual potential, also lives):

$$f(\tau_i, \xi_k) = \underbrace{\frac{\mu(t_{i+1}, \xi_k) - \mu(t_i, \xi_k)}{\Delta t_i}}_{\partial_t \mu} + \underbrace{\frac{\psi(\tau_i, x_k) - \psi(\tau_i, x_{k-1})}{\Delta x}}_{\partial_x \psi} - \varepsilon \underbrace{D_x^2 [\bar{I}_t \mu]}_{\text{diffusion}}(\tau_i, \xi_k), \quad (4)$$

where $\bar{I}_t \mu(\tau_i, \xi_k) := \frac{1}{2}[\mu(t_i, \xi_k) + \mu(t_{i+1}, \xi_k)]$ is exactly the $\hat{\rho}$ map of (1) (`interp_t_at_phi`, reused verbatim), and D_x^2 is the standard 3-point second difference in x (`deriv_x_at_phi` $\circ$ `deriv_x_at_m`, with the zero-Neumann ghost at $k = 1, n_x$ – entirely a space operator, untouched by any of this).

Every term in (4) is local to cell i in time. The $\partial_t \mu$ term only ever reads μ at i 's own two bounding edges, divided by i 's own width – it never sees a neighboring cell's width, so there is nothing to special-case at a junction. The diffusion term first applies $\bar{I}_t$ (Lemma 1: exact at τ_i for *any* Δt_i , independent of neighboring cells) and then a purely spatial, uniform- Δx operator. The $\partial_x \psi$ term never touches time at all. **Consequence:** the discrete FP constraint (4), as literally defined by this codebase, has the *same* local truncation order at every cell – including both refined end-cells and their neighbors – as it does on the uniform grid. Nothing degrades at the junction; the refinement changes the (local) resolution, not the (local) accuracy.

3.1 Worked instantiation, $n_t = 6$

Using the edges from Section 2.1, at the first junction ($i = 0, 1, 2$):

$$f(\tau_0, \xi_k) = \frac{\mu(\frac{1}{12}, \xi_k) - \rho_0(\xi_k)}{1/12} + \frac{\psi(\tau_0, x_k) - \psi(\tau_0, x_{k-1})}{\Delta x} - \varepsilon D_x^2 \left[\frac{1}{2} (\rho_0 + \mu(\frac{1}{12}, \cdot)) \right] (\xi_k), \quad (5)$$

$$f(\tau_1, \xi_k) = \frac{\mu(\frac{1}{6}, \xi_k) - \mu(\frac{1}{12}, \xi_k)}{1/12} + (\dots)/\Delta x - \varepsilon D_x^2 \left[\frac{1}{2} (\mu(\frac{1}{12}, \cdot) + \mu(\frac{1}{6}, \cdot)) \right] (\xi_k), \quad (6)$$

$$f(\tau_2, \xi_k) = \frac{\mu(\frac{1}{3}, \xi_k) - \mu(\frac{1}{6}, \xi_k)}{1/6} + (\dots)/\Delta x - \varepsilon D_x^2 \left[\frac{1}{2} (\mu(\frac{1}{6}, \cdot) + \mu(\frac{1}{3}, \cdot)) \right] (\xi_k), \quad (7)$$

and mirrored at the last junction ($i = 5, 6, 7$):

$$f(\tau_5, \xi_k) = \frac{\mu(\frac{5}{6}, \xi_k) - \mu(\frac{2}{3}, \xi_k)}{1/6} + (\dots), \quad (8)$$

$$f(\tau_6, \xi_k) = \frac{\mu(\frac{11}{12}, \xi_k) - \mu(\frac{5}{6}, \xi_k)}{1/12} + (\dots), \quad (9)$$

$$f(\tau_7, \xi_k) = \frac{\rho_1(\xi_k) - \mu(\frac{11}{12}, \xi_k)}{1/12} + (\dots). \quad (10)$$

Only the denominator of the $\partial_t \mu$ term changes across each row (per the local Δt_i); every other piece of every row has exactly the same form it would on the uniform grid.

4 The projection: exact enforcement via the Euclidean adjoint

The projection solves $f = 0$ *exactly* (up to linear-solve tolerance) by writing the correction in terms of a potential ϕ living at the same (τ_i, ξ_k) locations as f , and solving $RR^*\phi = f$ where R is the linear residual map (4) and R^* is *its Euclidean (array-index) adjoint* – not a physically/quadrature-weighted adjoint. This is a structural, algebraic fact, not an approximation: correcting $(\mu, \psi) \mapsto (\mu, \psi) + R^*\phi$ gives $R(\mu + R^*\phi\text{-part}, \psi + R^*\phi\text{-part}) = f - RR^*\phi = 0$ *exactly*, for *any* ϕ solving $RR^*\phi = f$, regardless of whether the individual pieces of R^* are themselves accurate interpolants of anything physical. This is why grid non-uniformity can never break *exactness* of the constraint enforcement: exactness only needs R^* to literally be R 's Euclidean transpose, which is a property that is checked once and for all (`scripts/validate_nu.py`'s adjoint-identity check, Section 6), not re-derived per grid.

4.1 `deriv_t_at_rho`: the one place the width jump appears in a formula

R^* 's time-derivative piece, ∂_t^* , is the adjoint of $\partial_t \mu$'s $1/\Delta t_i$ -weighted first difference:

$$(\partial_t^* v)_j = \frac{v_{j+1}}{\Delta t_{j+1}} - \frac{v_j}{\Delta t_j}. \quad (11)$$

At the fine/coarse junction $j = 1$ (interior node between cell 1, width $h/2$, and cell 2, width h , in the worked example):

$$(\partial_t^* v)_1 = \frac{v_2}{h} - \frac{2v_1}{h}, \quad (12)$$

a factor-of-2 asymmetry between the two terms coming purely from cell 1 being half the width of cell 2 – each term rescaled by its *own* cell's width, not by any shared center-to-center distance. Mirrored at $j = 5$: $(\partial_t^* v)_5 = 2v_6/h - v_5/h$.

4.2 interp_t_at_rho: an accuracy caveat (not a correctness one)

The other time-direction piece used inside the correction, `interp_t_at_rho` (feeding $\nabla_x \phi$'s contribution back into ρ_{out}), is the plain average $\frac{1}{2}(v_i + v_{i+1})$ mapping cell-center data back to the edge t_{i+1} between them. Unlike Lemma 1 (which places the average *at* the midpoint of the two sample points, exactly), this map estimates the value *at* t_{i+1} from samples at τ_i, τ_{i+1} – and t_{i+1} is the midpoint of $[\tau_i, \tau_{i+1}]$ only when $\Delta t_i = \Delta t_{i+1}$ (locally uniform). A Taylor expansion gives

$$\frac{1}{2}[g(\tau_i) + g(\tau_{i+1})] - g(t_{i+1}) = \frac{\Delta t_{i+1} - \Delta t_i}{4} g'(t_{i+1}) + O(\Delta t^2), \quad (13)$$

i.e. a first-order error proportional to the local width *mismatch*, vanishing whenever neighboring cells share a width. At the $n_t = 6$ junction ($j = 1$: $\Delta t_1 = h/2 \rightarrow \Delta t_2 = h$) this is $O(h)$, localized to that one edge's estimate. This does *not* affect whether the constraint $f = 0$ is satisfied after projection (Section 4 above – that's exact, algebraically) or the accuracy of the constraint itself (Section 3 – `interp_t_at_rho` never appears in f 's definition, only in the least-squares correction machinery); it is purely a statement that the *internal* quantity $\nabla_x \phi$ used to build that correction is a slightly less accurate estimate right at the two junction edges. Since ϕ itself carries no independent physical meaning (it is a Lagrange multiplier, not part of the state), this has no observable consequence beyond a highly localized constant in the projection's own internal bookkeeping.

5 What does not need re-deriving

- `projection_nu.py`'s per-mode tridiagonal operator (M_0, M_1, M_2) is assembled *numerically*, by probing `deriv_t_at_rho` / `interp_t_at_rho` / etc. with unit vectors, not from a closed-form formula. It automatically picks up whatever Δt_i the grid has, including the jumps above – no new algebra needed.
- `prox_nu.py`'s $\sigma_{\text{eff}} = \sigma \Delta t_i / \Delta t_{\text{ref}}$ weighting is already generic and diagonal in i (the KE prox is pointwise in (τ_i, ξ_k) , so it never sees a neighboring row's width at all). In the $n_t = 6$ example, $\Delta t_{\text{ref}} = 1/8$ (mean width of the actual 8-cell grid), so the fine end cells get $\sigma_{\text{eff}} = \sigma \cdot \frac{1/12}{1/8} = \frac{2}{3}\sigma$ and the coarse middle cells get $\sigma_{\text{eff}} = \sigma \cdot \frac{1/6}{1/8} = \frac{4}{3}\sigma$.
- `pipeline_nu.py`'s $\Delta t_j^{\text{dual}} = \frac{1}{2}(\Delta t_j + \Delta t_{j+1})$ quadrature weight for the staggered-grid norm is likewise generic; at the junction $\Delta t_1^{\text{dual}} = \frac{1}{2}(h/2 + h) = \frac{3}{4}h$.

6 Implementation

Added to `time_grid.py`:

```
def refine_ends_time_grid(nt: int) -> TimeGrid:
    """Uniform nt-cell grid with cell 0 and cell nt-1 each split into two
    equal halves. Returns a TimeGrid with nt+2 cells total."""
    if nt < 2:
        raise ValueError("nt_must_be_>=2_to_refine_both_ends_without_overlap")
    h = 1.0 / nt
    base_edges = np.linspace(0.0, 1.0, nt + 1)
    t_edges = np.concatenate([
```

```

        [base_edges[0], base_edges[0] + 0.5 * h],
        base_edges[1:-1],
        [base_edges[-1] - 0.5 * h, base_edges[-1]],
    ])
    return _time_grid_from_edges(t_edges)

```

Verified against Table 1 by hand: `refine_ends_time_grid(6)` reproduces the $t/\Delta t/\tau$ values above exactly.

Decisions made on the open design questions

1. **n_t is the base resolution**, so the returned grid has $n_t + 2$ cells, not n_t . This breaks the “constructor returns exactly n_t cells” pattern that `clustered_time_grid` has, but composes more cleanly ($n_t = 6$ unambiguously means “start from the 6-cell uniform grid and refine its ends”). $n_t < 2$ raises `ValueError` (the two end cells would coincide).
2. **Fixed single halving, no levels/k parameter yet** – kept to exactly what was asked (“start simple”). The name leaves room to add that later without an API break.
3. **Validation added**: `scripts/validate_nu.py`’s adjoint-identity check and repeated-projection stability check both now also run against `refine_ends_time_grid(NT)`, alongside the existing uniform/clustered cases. All checks pass (5/5). Not yet added: a dedicated `sb.gaussian_nonuniform.py`-style run against the closed-form SB solution on this specific grid.